Hoja de trabajo #2 — Transformers y mecanismos de atención

CC3092 Deep Learning y Sistemas Inteligentes · Javier Benítez (23405) · 5 de octubre de 2026

2. De las RNN a la atención

Cuello de botella. En un seq2seq con RNN el encoder comprime toda la oración en un único vector de tamaño fijo (el último estado oculto) y el decoder sólo ve ese vector. Mientras más larga la oración, más información hay que meter en el mismo espacio y lo primero que se leyó se "olvida", por lo que el BLEU cae con la longitud. **Bahdanau** conserva *todos* los estados del encoder y, en cada paso de decodificación, calcula pesos de alineación sobre ellos para construir un vector de contexto distinto por paso: el decoder "mira" directamente la parte relevante de la entrada.

Aditiva vs. multiplicativa. Bahdanau puntúa con una pequeña red: $e = v^T \tanh(W_1 s + W_2 h)$. Luong usa un producto punto $e = s^T h$ (o $s^T W h$). La multiplicativa es más eficiente en hardware moderno porque todos los puntajes salen de **una sola multiplicación de matrices** QK^T, operación altamente optimizada en GPU/TPU; la aditiva necesita una tanh elemento a elemento y otra proyección por cada par. Con d grande, la aditiva era algo mejor sin escalamiento, lo que motiva dividir entre $\sqrt{d_k}$.

Q, K, V como diccionario suave. Un diccionario normal compara la consulta con cada llave y devuelve *exactamente un* valor. La atención compara la consulta (query) con todas las llaves (keys) mediante similitud, convierte las similitudes en pesos con softmax y devuelve un **promedio ponderado** de los valores: es una búsqueda "suave" y diferenciable. En **self-attention** Q, K y V son proyecciones lineales (W_Q, W_K, W_V) de la misma secuencia; en **cross-attention** Q viene del decoder y K, V de la salida del encoder.

Limitaciones que elimina el Transformer. (1) Procesamiento secuencial: la RNN necesita h_{t-1} para calcular h_t, no se paraleliza en el tiempo; el Transformer procesa todos los tokens a la vez. (2) Dependencias largas: entre dos tokens a distancia n la señal recorre O(n) pasos en la RNN y O(1) en self-attention. (3) Desvanecimiento del gradiente: el gradiente ya no se multiplica n veces por el mismo Jacobiano; además las conexiones residuales dan un camino directo.

3. La arquitectura Transformer

Scaled dot-product attention. Si q_i, k_i son independientes con media 0 y varianza 1, $q \cdot k = \Sigma q_i k_i$ tiene media 0 y **varianza d_k** (suma de d_k términos de varianza 1). Sin escalar, los logits crecen como $\sqrt{d_k}$, la softmax se satura hacia un one-hot y su Jacobiano ($p_i(\delta_{ij}-p_j)$) se acerca a 0: los gradientes se desvanecen. Dividir entre $\sqrt{d_k}$ devuelve la varianza a 1. Lo verificamos empíricamente (Fig. 1).

Multi-head. h cabezas de dimensión d_{model}/h permiten atender a la vez a distintos subespacios y relaciones (sintaxis, posición, correferencia) en lugar de promediarlas en una sola distribución. El número de parámetros **no cambia** ($4 \cdot d_{model}^2$ + sesgos; comprobado: 1,050,624 para d=512 con h=1,2,4,8) y el costo es similar.

Codificación posicional. Self-attention es *equivariante a permutaciones*: si se reordenan los tokens, la salida se reordena igual, por lo que no sabe el orden. La versión sinusoidal suma $PE(pos,2i)=\sin(pos/10000^{2i/d})$, $PE(pos,2i+1)=\cos(\cdot)$: no tiene parámetros, cada dimensión es una frecuencia y PE(pos+k) es función lineal de PE(pos). Los **embeddings aprendidos** (BERT, GPT-2) son una tabla entrenable, flexibles pero limitados a la longitud máxima vista. **RoPE** rota q y k según su posición, de modo que $q \cdot k$ depende sólo de la distancia relativa; generaliza mejor a contextos largos (LLaMA y modelos actuales).

Bloque completo. Cada subcapa (atención y FFN) va envuelta en una conexión residual x + Sublayer(x), que facilita el flujo del gradiente, y LayerNorm, que estabiliza la escala de activaciones. La FFN (dos capas lineales con ReLU/GELU, $d_{ff}=4d$) se aplica igual a cada posición y aporta la no linealidad y "memoria". **Post-LN** (original): LN(x + Sublayer(x)); necesita warm-up y es inestable en redes profundas porque los gradientes cerca de la salida son grandes. **Pre-LN**: x + Sublayer(LN(x)); el camino residual queda limpio, entrena estable sin warm-up y es el estándar actual (`norm_first=True` en PyTorch).

Complejidad. Self-attention cuesta **$O(n^2 d)$ en tiempo y $O(n^2)$ en memoria** (matriz n×n por cabeza). Alternativas: **FlashAttention** calcula exacto por bloques en SRAM sin materializar la matriz (memoria O(n), mucho más rápido) pero sigue siendo $O(n^2)$ en cómputo y requiere kernels específicos; **atención por ventanas** (Longformer, Mistral) limita a w vecinos, O(nw), sacrificando el contexto global directo; **atención lineal** (Performer, Linear Transformer) aproxima softmax con kernels y logra O(n), sacrificando exactitud/calidad.

4. Atención en PyTorch y Hugging Face

Clase / función	Propósito y parámetros relevantes
`F.scaled_dot_product_attention`	Calcula $\text{softmax}(QK^T/\sqrt{d})V$ con kernels fusionados (Flash / memory-efficient). *attn_mask*: bool (True=permitido) o float sumado a los logits; *dropout_p*: dropout sobre los pesos; *is_causal*: aplica máscara triangular; *scale*: reemplaza $1/\sqrt{d}$.
`nn.MultiheadAttention`	Proyecciones Q/K/V/O + h cabezas. *embed_dim*, *num_heads* (embed_dim divisible), *batch_first* (B,n,d), *kdim/vdim* para K/V de otra dimensión (cross-attention). forward: *key_padding_mask* (B,m) ignora padding; *attn_mask* (n,m) p.ej. causal; *need_weights* devuelve pesos; *average_attn_weights*=False los da por cabeza.

`nn.TransformerEncoderLayer`	Bloque self-attn + FFN con residuales y LayerNorm. *d_model*, *nhead*, *dim_feedforward* (FFN), *dropout*, *norm_first* (True = Pre-LN). `nn.Transformer.generate_square_subsequent_mask(n)` crea la máscara causal float con -inf arriba de la diagonal.
Hugging Face `output_attentions`	Con *output_attentions=True* (y *attn_implementation='eager'*, ya que SDPA no materializa pesos) el modelo devuelve una tupla con una matriz por capa de forma (batch, cabezas, n, n); en BERT/GPT-2 base: 12 × (1,12,n,n).

Procedimiento: ejercicio a mano

Tres tokens, d_k=2, Q=K=[[1,0],[0,1],[1,1]], V=[[1,0],[0,2],[1,1]]. (1) $S=QK^T$=[[1,0,1],[0,1,1],[1,1,2]]. (2) $S/\sqrt{2}$=[[.707,0,.707],[0,.707,.707],[.707,.707,1.414]]. (3) Softmax por fila: fila 1: $e^{.707}$=2.028, e^0=1, suma 5.056 → (0.401, 0.198, 0.401); fila 2 → (0.198, 0.401, 0.401); fila 3: 2.028, 2.028, $e^{1.414}$=4.113, suma 8.169 → (0.248, 0.248, 0.504). (4) Salida AV: (0.802, 0.797), (0.599, 1.203), (0.752, 1.000). (5) Con máscara causal: fila 1 → (1,0,0); fila 2 → softmax(0, .707)=(0.330, 0.670) → (0.330, 1.340); fila 3 igual. El notebook reproduce exactamente estos valores con `F.scaled_dot_product_attention` (con y sin *is_causal*).

6.1 Verificaciones, escalamiento y costo computacional

Se implementó `attention()` y `MultiHeadAttention` desde cero. Resultados: (a) vs. `F.scaled_dot_product_attention`: max|dif| = 2.4×10^{-7} sin y con máscara causal; (b) vs. `nn.MultiheadAttention` copiando los mismos pesos y usando `generate_square_subsequent_mask`: max|dif| = 8.9×10^{-8} en salida y en pesos por cabeza; (c) todas las filas de pesos suman 1 y con máscara causal la parte estrictamente superior es exactamente 0; (d) el número de parámetros es idéntico para h = 1, 2, 4, 8; (e) Var(q·k) empírica = 4.0, 16.0, 63.7, 262, 1014 para d_k = 4…1024 (≈ d_k).

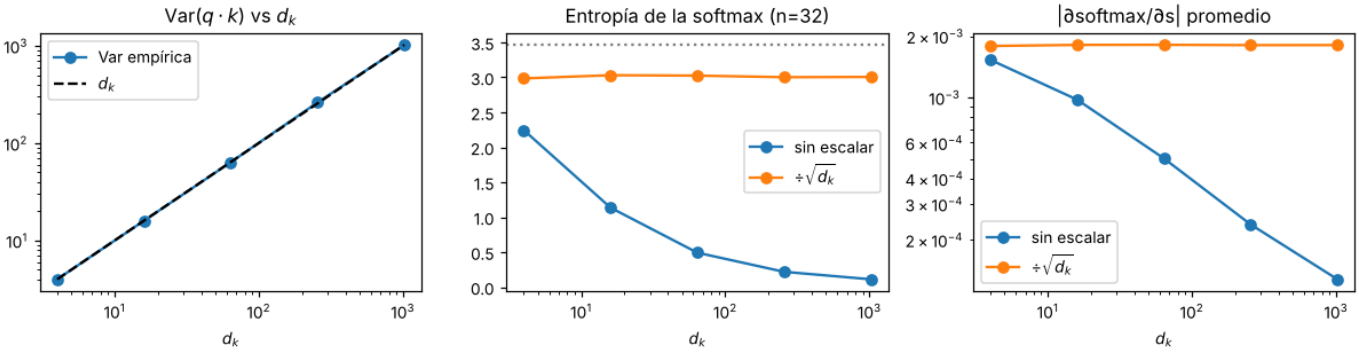


Fig. 1. Izq.: Var(q·k) crece linealmente con d_k. Centro/der.: sin escalar, la entropía de la softmax (n=32; máx. log 32 ≈ 3.47) y la magnitud del Jacobiano colapsan al crecer d_k; con ÷√d_k se mantienen constantes.

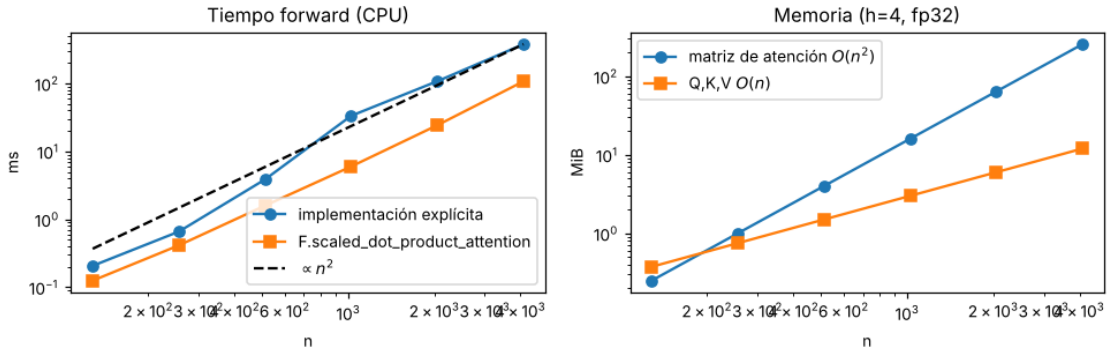


Fig. 2. Costo computacional (CPU, h=4, d=64). El tiempo de la implementación explícita sigue ~n^2 (0.2 ms → 378 ms de n=128 a 4096); la versión fusionada de PyTorch es 3–5× más rápida pero con la misma pendiente. La matriz de atención domina la memoria: para n=100,000 serían 37 GiB por cabeza en fp32.

5. Visualización de atención en modelos preentrenados

Se seleccionó automáticamente, entre las 144 cabezas de cada modelo, la que mejor representa cada patrón para la oración *"The cat sat on the mat because it was tired."* (Fig. 3).

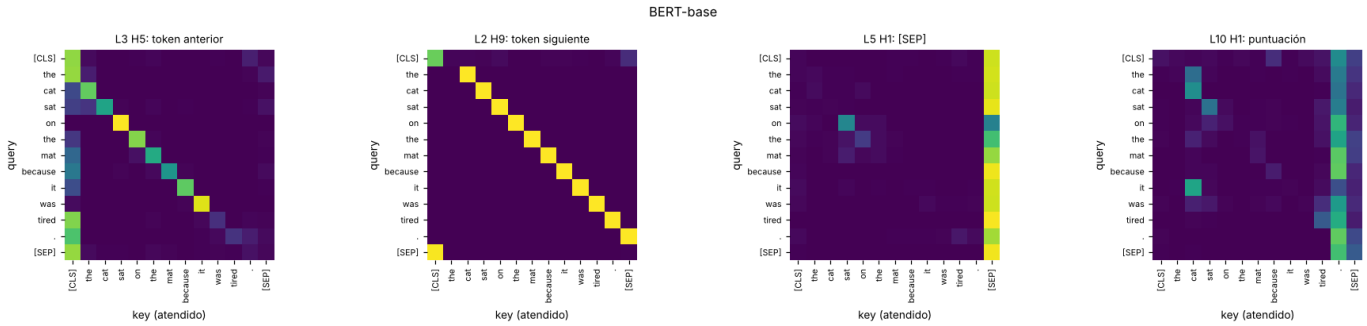


Fig. 3a. BERT: L3H5 atiende al token anterior, L2H9 al siguiente (0.92 de la masa en la diagonal desplazada), L5H1 vuelca casi todo en [SEP] (0.88) y L10H1 en la puntuación ".".

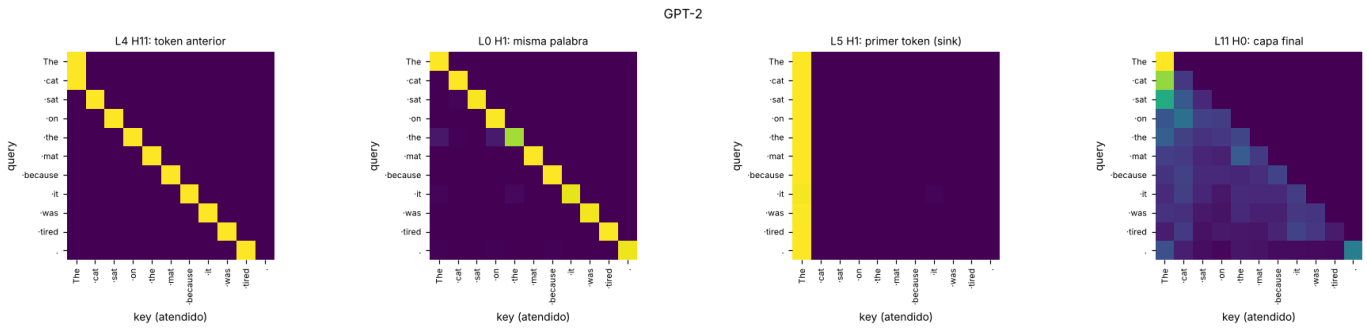


Fig. 3b. GPT-2: L4H11 token anterior, L0H1 la misma palabra, L5H1 casi el 100 % en el primer token (attention sink) y L11H0 un patrón difuso. Todas son triangulares inferiores.

Correferencia (it → animal/street). Con el par "…because it was too *tired*/*wide*." se buscó la cabeza de BERT que maximiza $[\text{att}(\text{it}\to\text{animal}) - \text{att}(\text{it}\to\text{street})]_{\text{tired}} + [\text{att}(\text{it}\to\text{street}) - \text{att}(\text{it}\to\text{animal})]_{\text{wide}}$. **L6H10** cumple: con *tired*, 0.28 a animal vs. 0.07 a street; con *wide*, 0.01 a animal vs. **0.59** a street (Fig. 4). Sólo 22 de 144 cabezas cumplen ambas desigualdades, y otras (L8H10, L9H0) atienden a *animal* en los dos casos.

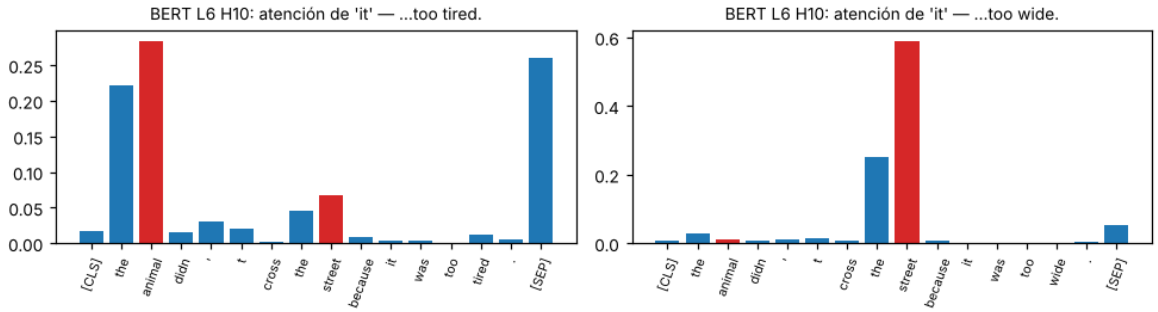


Fig. 4. Distribución de atención del token "it" en BERT L6H10 para ambas oraciones (rojo: animal y street).

GPT-2: máscara causal y attention sinks. En las 12×12 matrices el máximo por encima de la diagonal es exactamente 0.0: son triangulares inferiores. La proporción de atención que recibe el primer token (excluyendo la query 0) crece con la profundidad: 0.26 (capa 0), 0.45 (1), 0.60 (4), 0.74 (5), 0.81 (7), 0.85 (10), 0.71 (11). Cambiando la primera palabra ("Yesterday…") el valor promedio en capas 2–11 sigue siendo 0.69, así que depende de la *posición*, no del contenido. Es el fenómeno de **attention sinks** (Xiao et al., 2023): como la softmax obliga a repartir masa que sume 1, las cabezas que "no necesitan" atender a nada la descargan en el primer token, visible para todas las queries; por eso StreamingLLM conserva esos tokens iniciales en la caché al usar ventanas. En BERT ocurre algo análogo con [CLS] en capas 1–2 (~0.4) y con [SEP] en capas 4–9 (~0.55).

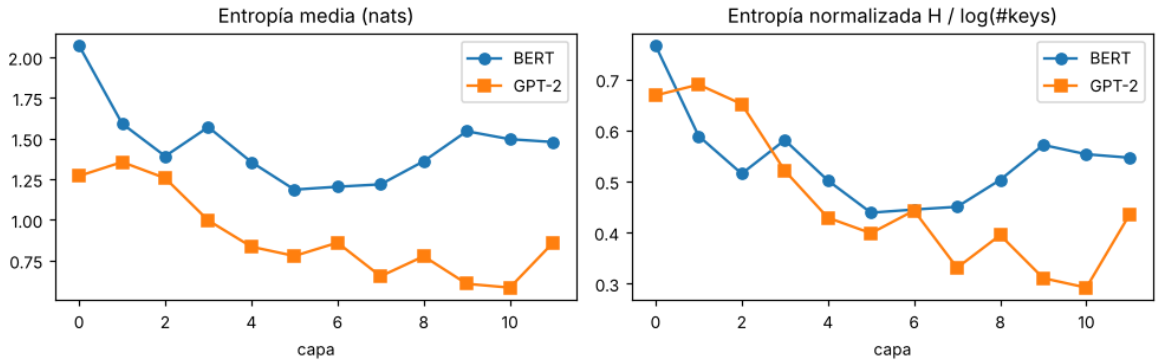


Fig. 5. Entropía media de la atención por capa (promedio de 4 oraciones). Der.: normalizada por log(#keys visibles) para comparar la máscara causal de GPT-2 con BERT.

Entropía. En BERT la entropía cae de 2.07 nats (capa 0, atención muy difusa) a ~1.2 en las capas medias (5–7, donde dominan los sinks en [SEP] y patrones posicionales) y vuelve a subir a ~1.5 en las capas 9–11. En GPT-2 baja casi monótonamente de 1.27 a ~0.6 en las capas 9–10: las capas profundas están **más concentradas**, pero sobre todo porque concentran la masa en el primer token (sink), no porque cada token elija una palabra semántica.

7. Discusión y análisis

¿Por qué la atención permitió modelos más grandes? (1) Paralelismo: todos los tokens de la secuencia se procesan con multiplicaciones de matrices en un solo paso, aprovechando GPU/TPU, mientras la RNN tiene n pasos dependientes. (2) Longitud de camino O(1) entre cualquier par de tokens, en vez de O(n), lo que facilita aprender dependencias largas y evita el desvanecimiento. (3) El costo cuadrático es el precio: para contextos de cientos o pocos miles de tokens, n^2d en paralelo es más barato en tiempo real que nd secuencial, así que escalar parámetros y datos resultó rentable.

¿Las cabezas se especializan? Sí: encontramos cabezas posicionales casi deterministas (siguiente token: 0.92), cabezas de delimitadores/puntuación, una cabeza de correferencia (L6H10) y en GPT-2 cabezas de "misma palabra" y de token anterior. Pero no todas parecen útiles: muchas cabezas de las capas medias de BERT y la mayoría de GPT-2 ponen >80 % de la masa en [SEP] o en el primer token, comportándose como una operación "no-op". Esto coincide con Clark et al. (2019) y Michel et al. (2019), donde gran parte de las cabezas se puede podar con poca pérdida.

¿Los pesos de atención explican las decisiones? Sólo parcialmente. Jain y Wallace (2019) muestran que los pesos se correlacionan poco con medidas de importancia por gradiente y que existen distribuciones de atención muy distintas con la misma predicción. Wiegreffe y Pinter (2019) responden que eso no prueba que no expliquen nada: las alternativas adversarias se construyen por instancia y, si se entrena el modelo completo, la atención aprendida sí aporta. Nuestros resultados apoyan una postura intermedia: L6H10 sugiere correferencia, pero sólo 22/144 cabezas la reflejan y otras atienden a *animal* en ambos casos; además la masa en sinks no tiene significado semántico y el peso ignora la norma de los vectores valor. La atención es una *pista* plausible, no una explicación fiel; para afirmarlo hacen falta pruebas causales (ablación, gradientes).

¿Cuándo preferir una RNN o CNN? Streaming en tiempo real o dispositivos con poca memoria (la RNN tiene estado O(1) por paso), series temporales y audio de entrada continua, datasets pequeños donde el sesgo inductivo de localidad (CNN) o de secuencia (RNN) generaliza mejor, visión con pocos datos, y secuencias muy largas donde los modelos de estado (Mamba, RWKV) son lineales.

Documentos de 100,000 tokens. La matriz n×n tendría 10^{10} entradas por cabeza (≈37 GiB en fp32; ×12 cabezas ×12 capas), y el tiempo crece 100^2 veces respecto a 1,000 tokens; además los embeddings posicionales aprendidos no extrapolan. Alternativas: FlashAttention (exacta, memoria lineal), atención por ventanas o dispersa (Longformer, BigBird), atención lineal (Performer), RoPE con interpolación/YaRN para extender el contexto, modelos de estado (Mamba), o recuperación (RAG) y fragmentación jerárquica para procesar sólo lo relevante.

Conclusiones

La atención escalada sustituye la recurrencia por una búsqueda suave paralelizable; el factor $1/\sqrt{d_k}$ es necesario para que la softmax no se sature ($\mathrm{Var}(q\cdot k)=d_k$, verificado). Nuestra implementación coincide con PyTorch a precisión de máquina y el costo medido confirma el crecimiento cuadrático. En los modelos preentrenados las cabezas muestran especialización (posicional, delimitadores, correferencia), pero gran parte de la atención se descarga en sinks ([SEP] en BERT, primer token en GPT-2), por lo que los mapas de atención deben interpretarse con cautela.

Referencias

Vaswani, A. et al. (2017). Attention Is All You Need. arXiv:1706.03762.
Bahdanau, D., Cho, K., Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR.
Luong, M.-T., Pham, H., Manning, C. (2015). Effective Approaches to Attention-based NMT. EMNLP.
Xiao, G. et al. (2023). Efficient Streaming Language Models with Attention Sinks. arXiv:2309.17453.
Jain, S., Wallace, B. (2019). Attention is not Explanation. NAACL. · Wiegreffe, S., Pinter, Y. (2019). Attention is not not Explanation. EMNLP.

Clark, K. et al. (2019). What Does BERT Look At? · Michel, P. et al. (2019). Are Sixteen Heads Really Better than One? NeurIPS.
Dao, T. et al. (2022). FlashAttention. NeurIPS · Su, J. et al. (2021). RoFormer (RoPE) · Xiong, R. et al. (2020). On Layer Normalization in the Transformer Architecture.
Documentación de PyTorch (torch.nn, torch.nn.functional) y Hugging Face Transformers.
Repositorio: https://github.com/JavierBenitez112/HDT2-Transformers